

Revision 10-13 • One Token End to End

Interactive revision of lessons 10, 11, 12 and 13: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 • Logits shape and a broken decode loop

MISSED

Covers: 10, 13

QUESTION

`out = model(input_ids)` with `input_ids` of shape $(1, T)$, then `next_id = out.logits[:, 0, :].argmax(-1)`. What's the logits shape? Why does it generate nonsense that ignores the prompt? How big are full logits for an 8,192-token prompt in FP32, and why do engines avoid them?

Answer

- **Shape = (batch, positions, vocab) = (1, T, 128,256):** at every position, a score for every vocabulary token ("what comes after the text up to here?").
- **Bug:** `logits[:, 0, :]` is the prediction after only the first token (BOS), so it ignores the prompt. Use the **last** position: `logits[:, -1, :]`.
- **Size:** $8,192 \times 128,256 \times 4 \text{ B} \approx 4.2 \text{ GB}$, vs **0.5 MB** for the last row. Engines run the LM head on the last position only and skip the rest.

RE-DRILL

Picture the tensor's dimensions before indexing: which axis is position, and which position predicts the next token?

Q2 • Why $-\infty$ in the mask, and why old rows never change

SOLID

Covers: 11

QUESTION

Future positions are masked by adding $-\infty$ before softmax: why $-\infty$ and not 0? In decode, a new token adds one row to the score grid: why do earlier rows never need recomputing?

Answer

- **$-\infty$:** $e^{(-\infty)} = 0$, so future tokens get exactly zero weight. A score of 0 would still get $e^0 = 1$, a real share, so the token would peek at the future.

- **Old rows:** each earlier row belongs to an earlier token and, under the causal mask, only sees tokens up to itself. The new token is in every earlier row's masked future, so it can't change them: add one row, reuse the rest (and cache their K, V).

Q3 · Repeating stories vs bizarre words PARTIAL

Covers: 13

QUESTION

A story assistant at temperature 0 (greedy) repeats the same phrase in long stories. Switching to temperature 1.2 with no other settings gives occasional bizarre words. Cause and fix for each?

Answer

- **Repetition (greedy):** always taking the top token locks open-ended text into loops. Fix: **sample** (temperature > 0); a repetition penalty helps too.
- **Bizarre words (T = 1.2, no cut):** temperature flattens the distribution, so the ~128k unlikely tokens share more probability, and over many tokens one eventually gets drawn. Fix: **cut the tail** with top-p or min-p.
- **Sampler order:** penalties → **temperature** → **cut the tail** → rescale → draw.

Q4 · Residual stream and what RMSNorm normalizes PARTIAL

Covers: 12

QUESTION

Why does each layer do $x \leftarrow x + f(x)$ instead of $x \leftarrow f(x)$? Does RMSNorm normalize the stream itself or something else, and why does it matter?

Answer

- **Residual benefits:** nothing overwrites the stream, so early information survives to the end; each layer only learns an **edit**; gradients flow straight through the "+" (no vanishing).
- **RMSNorm normalizes a copy:** before attention and before the MLP, a **copy** of the stream is normalized and fed in; the result is added back to the **raw** stream.
- **Why it matters:** after 64 additions the stream's scale drifts, and sublayers need inputs of predictable size (~1). The stream itself keeps its raw values, so nothing accumulated gets squashed. Growing hidden-state values are normal; a final norm comes before the LM head.

Exercise · Build the sampler PARTIAL

Covers: 13

QUESTION

From raw logits: greedy if $T = 0$, else temperature $\rightarrow$ softmax $\rightarrow$ cut the tail (top-p, min-p) $\rightarrow$ rescale $\rightarrow$ draw. Toy logits 5.0, 3.2, 2.9, 2.1, 0.5 (Paris, the, a, located, Lyon).

```
def sampler_probs(logits, temperature=1.0, top_p=1.0, min_p=0.0):
    n = len(logits)
    if temperature == 0:                                # 1 · greedy
        best = max(range(n), key=lambda i: logits[i])
        return [1.0 if i == best else 0.0 for i in range(n)]
    scaled = [logits[i] / temperature for i in range(n)] # 2 · temperature
    m = max(scaled)                                     # 3 · softmax
    exps = [math.exp(x - m) for x in scaled]
    probs = [x / sum(exps) for x in exps]
    order = sorted(range(n), key=lambda i: probs[i], reverse=True) # 4a · top-p
    keep, running = set(), 0.0
    for i in order:
        running += probs[i]
        keep.add(i)                                     # add every token walked past...
        if running >= top_p:                             # ...stop once the running sum reaches top_p
            break
    keep = [i for i in keep if probs[i] >= min_p * max(probs)] # 4b · min-p
    total = sum(probs[i] for i in keep)                  # 5 · rescale
    return [probs[i] / total if i in keep else 0.0 for i in range(n)]
```

Settings	Result
$T = 1$, no cut	0.74, 0.12, 0.09, 0.04, 0.01
$T = 0.5$	Paris 0.96
$\text{top}_p = 0.9$ / $\text{min}_p = 0.1$	0.78, 0.13, 0.10, 0, 0
$T = 0$	Paris 1.0
10,000 draws at $\text{top}_p = 0.9$	0.78, 0.13, 0.09